

Project Title: Threat Intelligence System

Course: CCP / Software Engineering Project

University: Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology

Department: Software Engineering

Submitted By (Group Members):

- Muhammad Madni
- Wasif Ali Raza
- Daud Latif
- Zohaib Luqman
- Haneesa awan

CCP Project Report

Threat Intelligence System (Spring Boot + Kafka + MySQL)

Technology Stack: Spring Boot, MySQL, Apache Kafka, HTML, Maven

Submission Type: Final Project Report + Source Code (GitHub)

GITHUB LINK:

<https://github.com/MuhammadMadni1607/Threat-Intelligence-System.git>

1. SRS (Software Requirements Specification)

1.1 Introduction

The Threat Intelligence System is a web-based application designed to collect, store, and manage Indicators of Compromise (IOC) such as IP addresses, domains, severity levels, and sources. It helps in detecting and analyzing cybersecurity threats using a centralized system.

1.2 Purpose

The purpose of this system is to:

- Store threat intelligence data
 - Provide REST APIs for data access
 - Process data using Kafka messaging system
 - Display data on a simple web dashboard
-

1.3 Scope

The system includes:

- IOC data management
 - MySQL database storage
 - Kafka-based message streaming
 - REST API backend
 - Simple frontend dashboard
-

1.4 Functional Requirements

- Add IOC data (IP, domain, severity, source)
 - Retrieve all IOC records
 - Store data in database
 - Send messages to Kafka topic
 - Display data in frontend
-

1.5 Non-Functional Requirements

- Fast response time
 - Scalable architecture
 - Reliable data storage
 - Secure communication
 - Easy user interface
-

1.6 Technologies Used

- Java (Spring Boot)
 - Spring Data JPA
 - MySQL Database
 - Apache Kafka
 - HTML, CSS, JavaScript
 - Maven
-

2. PROJECT DOCUMENTATION

2.1 Overview

This project is a full-stack threat intelligence system that processes cybersecurity data using Spring Boot, MySQL, and Kafka. It supports real-time message streaming and data storage.

2.2 Working Flow

1. User submits IOC data
 2. Data is stored in MySQL
 3. Kafka producer sends message to topic
 4. Kafka consumer reads message
 5. Data is shown in dashboard
-

3. SYSTEM OVERVIEW

The system follows a layered architecture:

Frontend (HTML Dashboard)



Controller Layer (Spring Boot REST API)



Service Layer



Repository Layer



MySQL Database



Kafka Messaging System

4. FEATURES

- Add IOC data (IP, Domain, Severity, Source)
 - View all threat records
 - Real-time Kafka messaging
 - MySQL database storage
 - Simple web dashboard
 - REST API support
-

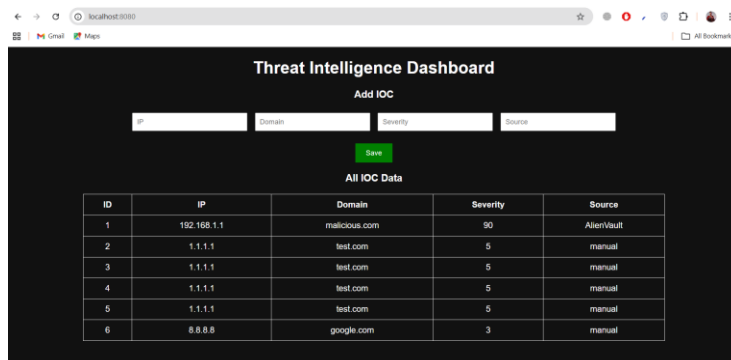
5. DATABASE DETAILS

Table: ioc_data

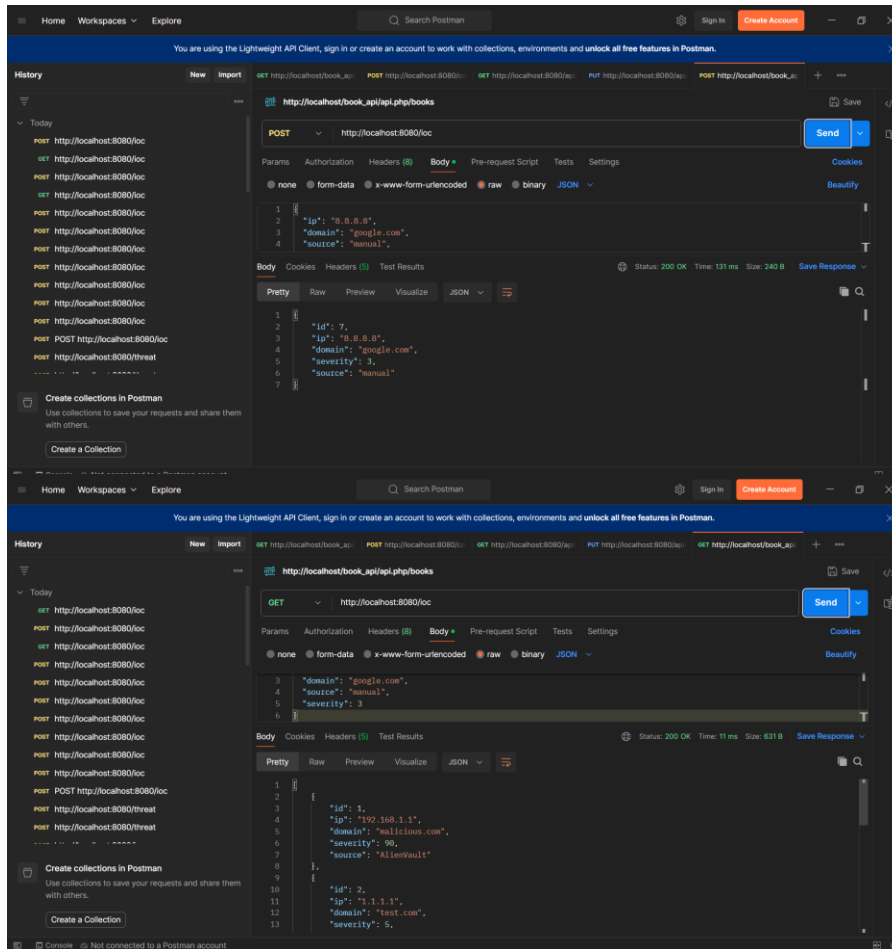
Field	Type	Description
id	Long	Primary Key
ip	String	IP Address
domain	String	Domain Name
severity	int	Threat Level
source	String	Data Source

6. SCREENSHOTS

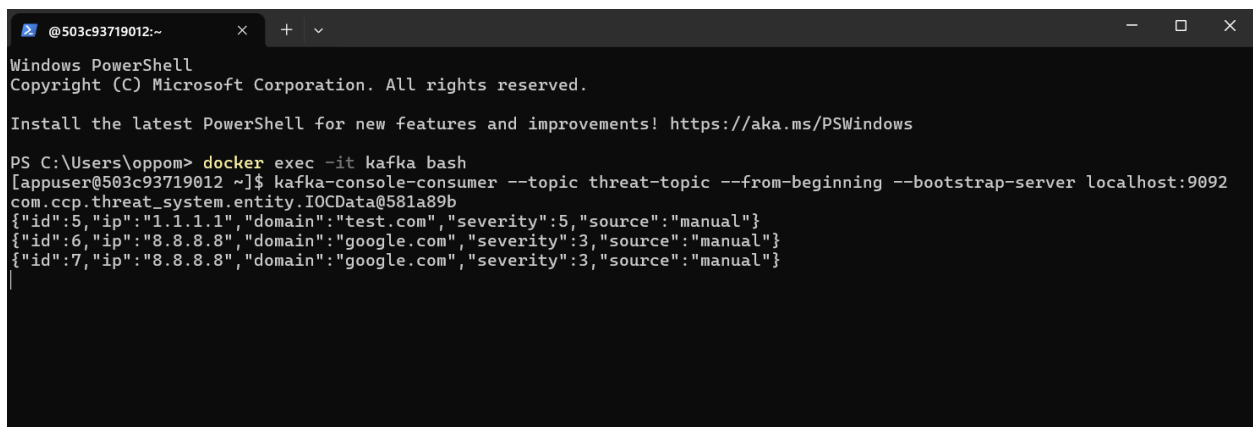
- Dashboard UI screenshot



- Postman API testing



- Kafka console output



- MySQL database table view

Query 1

```
1 CREATE DATABASE ccp_threat_db;
2 USE ccp_threat_db;
3
4 SHOW TABLES;
5 USE ccp_threat_db;
6
7 SELECT * FROM ioc_data;
```

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid

	id	domain	ip	severity	source
▶	1	malicious.com	192.168.1.1	90	AlienVault
	2	test.com	1.1.1.1	5	manual
	3	test.com	1.1.1.1	5	manual
	4	test.com	1.1.1.1	5	manual
	5	test.com	1.1.1.1	5	manual
	6	google.com	8.8.8.8	3	manual
	7	google.com	8.8.8.8	3	manual

ioc_data 5

Output

Action Output

#	Time	Action	Message	Duration / Fetch
2	19:51:55	SHOW TABLES	Error Code: 1046. No database selected Select the default DB to be used by double-clicking its name in the SC...	0.000 sec
3	19:51:57	USE ccp_threat_db	0 row(s) affected	0.000 sec
4	19:51:59	SHOW TABLES	1 row(s) returned	0.000 sec / 0.000 sec
5	19:55:48	SELECT * FROM ioc_data LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
6	21:19:48	SELECT * FROM ioc_data LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
7	21:55:01	SELECT * FROM ioc_data LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
8	22:20:38	SELECT * FROM ioc_data LIMIT 0, 1000	7 row(s) returned	0.000 sec / 0.000 sec

7. CONCLUSION

The Threat Intelligence System successfully demonstrates a real-world cybersecurity data processing system using Spring Boot, MySQL, and Kafka. It provides efficient storage, processing, and visualization of threat data. This project can be extended in future with AI-based threat detection and real-time monitoring dashboards.